

Project Workbook
Vinilos
(Diego, Pilar y Javier)
1º DAM/DAW 2025/2026

WORKBOOK INDEX

1. Project Introduction
2. Project Objectives
3. System Overview
4. Functional Requirements
5. Non-Functional Requirements
6. Technologies Used
7. System Architecture
8. Database Design
9. Backend Modeling
10. Backend Implementation
11. Frontend Modeling
12. Frontend Implementation
13. System Testing
14. User Manual
15. Project Conclusions
16. Bibliography / Sources

INTRODUCTION

The VinylMarket project is a web application developed as part of the Cross-Platform Application Development (DAM) module. Its main objective is to create a modern platform that allows users to search for, manage and store information about vinyl records, combining its own data with information obtained from the Discogs public API, one of the world's largest music databases.

The application consists of two main parts:

A backend developed in Java using Spring Boot, responsible for managing the system logic, the connection to the MySQL database and communication with the external Discogs API.

A frontend built with HTML, CSS and JavaScript, which provides a clear and functional interface where the user can interact with the catalogue, perform searches, log in and manage their personal collection.

The project aims to simulate the operation of a real vinyl collectors' platform, allowing users to:

- View real vinyl records from Discogs.
- Importing them into the user's own catalogue.

- Saving them to the user's personal collection.
- Viewing them on a dynamic dashboard.
- Managing users via registration and login.

In addition to meeting the module's technical requirements, the project serves as a comprehensive exercise in integrating frontend, backend, database and external services, replicating a professional development environment.

Project objectives

The VinylMarket project aims to develop a comprehensive web application that enables users to manage a vinyl catalogue and their personal collection, integrating their own data with information obtained from the external Discogs API. To this end, the following objectives have been set:

Overall objective

To develop a functional web platform that enables users to search for, view, import and manage music vinyl records, combining a Java backend using Spring Boot, a MySQL database and a dynamic frontend built using HTML, CSS and JavaScript.

Specific objectives

- Implement a REST backend using Spring Boot to manage users, records and collections.
- Connect to the Discogs API to retrieve real-time information about vinyl records.
- Design and create a MySQL database to store users, records and personal collections.
- Develop an interactive frontend that allows users to browse, search for and import vinyl records.
- Implement an authentication system with registration and login.
- Create a dynamic dashboard displaying records from the catalogue and the personal collection.
- Manage frontend-backend communication using the Fetch API.
- Ensure data persistence using JPA and repositories.
- Resolve real-world integration issues, such as CORS, fetch errors, data structure or endpoints.
- Document the development process to produce the Workbook.

Learning objectives

- Understand MVC architecture in a real-world project.
- Learn how to call external APIs from a backend.
- Handle HTTP requests (GET, POST) from the frontend.
- Integrate multiple technologies within a single project.
- Develop debugging and troubleshooting skills.
- Work with JSON, REST controllers and services.

System Overview

The VinylMarket system is a web application designed to manage a catalogue of music vinyl records and allow users to create their own personal collection. The system combines information stored in its own database with data obtained in real time from the Discogs public API, enabling it to work with actual vinyl records, authentic cover art and official metadata.

The system is divided into two main components:

- **Backend (REST API with Spring Boot)**

The backend is the logical core of the system. It is responsible for:

- Managing users (registration, login, authentication)
- Querying the Discogs API to retrieve actual vinyl records
- Saving records to the MySQL database
- Managing each user's personal collection
- Exposing REST endpoints so that the frontend can interact

The architecture follows the MVC pattern:

Model → Classes such as Record, User, Collection

Repository → Access to MySQL via JPA

Service → Business logic (Discogs, catalogue, collections)

Controller → REST endpoints accessible from the frontend

The backend acts as an intermediary between the database, Discogs and the web interface.

- **Frontend (HTML, CSS and JavaScript)**

The frontend is the visual part of the system. It allows the user to:

- Search for actual vinyl records on Discogs
- Import them into their own catalogue
- View their personal collection
- Navigate the dashboard
- Log in and register
- View album covers, titles, artists and genres
- Communication with the backend is handled via the Fetch API, utilising the REST endpoints created in Spring Boot.

The main app.js file manages:

- Calls to /api/vinilos/search
- Importing vinyl records
- Dynamic rendering of cards
- User management in localStorage
- Updating the dashboard

- **Database (MySQL)**

- The system uses a relational database with three main tables:
- users → login and profile details
- records → general vinyl catalogue
- collection → user–record relationship

The database ensures data persistence and allows each user to have their own personalised collection.

- **Integration with Discogs**

One of the most important elements of the system is the integration with the Discogs API, which allows:

- Searching for real vinyl records by title or artist
- Retrieving cover art, genres, year and metadata
- Importing records into the user's own catalogue

The DiscogsService handles making HTTP requests, processing the received JSON and converting it into Disc objects.

- **General operation**

- The user logs in or registers.
- From the dashboard, they can search for real vinyl records.
- The backend queries Discogs and returns the results.
- The user can import a vinyl record into the catalogue.
- The vinyl record is saved in MySQL.
- The user can add it to their personal collection.
- The dashboard displays the available vinyl records and the user's collection.

Functional requirements

Functional requirements describe what the system is expected to do. In your VINILOS project, they are divided into four sections: users, catalogue, Discogs and personal collection.

- **User management**

1. The system must allow new users to be registered.
2. The system must allow users to log in using their email address and password.
3. The session must be maintained using localStorage.
4. Duplicate email addresses must not be permitted.
5. Users must be able to log out via the interface.

- **Vinyl catalogue management**

6. List all vinyl records stored in the database.
7. View detailed information about a vinyl record.
8. Save a vinyl record to the general catalogue.

9. Avoid duplicates using the discogsId field.
10. Display the catalogue on the user's dashboard.
 - **Integration with the Discogs API**
11. Search for actual vinyl records using the Discogs API.
12. Retrieve actual data: title, artist, genre, year, cover art.
13. Convert the JSON response into Disco objects.
14. Import a vinyl record from Discogs into your own catalogue.
15. Handle errors when Discogs does not return any results.
 - **Personal collection management**
16. Add a record from the catalogue to the user's collection.
17. Prevent a user from adding the same record twice.
18. List the records in the personal collection.
19. Display the collection on the dashboard.
20. Link each record to a user via the 'collection' table.
 - **User interface and experience**
21. Display Discogs results in visual cards.
22. Display the general catalogue on the dashboard.
23. Display the user's personal collection.
24. Allow vinyl records to be imported via front-end buttons.
25. Allow vinyl records to be added to the collection from the dashboard.

No functional requirements

Non-functional requirements describe how the system should behave; in other words, the quality characteristics it must meet to ensure it operates correctly, safely and efficiently.

They are divided into performance, security, usability, availability and maintainability.

- **Performance**

1. The system must respond to user requests within 2 seconds for common operations (search, listing, login).
2. The dashboard must load within 3 seconds on a standard connection.
3. Requests to the Discogs API must be handled efficiently to avoid crashes or prolonged delays.

- **Security**

4. Passwords must be stored in encrypted form in the database.
5. Private keys and Discogs tokens must not be exposed on the frontend.
6. The system must validate all data received to prevent SQL injections or similar attacks.
7. Sensitive operations must require an authenticated user.

- **Usability**

8. The interface must be clear, intuitive and easy to navigate.
9. Action buttons (import, add to collection, search) must be visible and accessible.
10. The system must display error messages that are clear to the user.
11. The design must be responsive and adapt to different screen sizes.

- **Availability**

12. The backend must be available whilst the server is running, without any unexpected interruptions.
13. The database must be accessible throughout the user's session.
14. The system must handle Discogs outages by displaying appropriate messages without blocking the application.

- **Maintainability**

15. The code must be organised according to the MVC architecture.
16. Controllers, services and repositories must be kept separate to facilitate modifications.
17. The project must allow new features to be added without affecting existing ones.
18. The code must be readable and commented where necessary.

- **Compatibility**

19. The system must function correctly in modern browsers (Chrome, Edge, Firefox).
20. The backend must be compatible with MySQL and Java 17.
21. The frontend must function without the need for additional plugins.

Technologies used

The VinylMarket project combines various technologies on both the backend and frontend, as well as development tools and external services. All of these work together to create a comprehensive, fully functional application connected to a live API.

- **Backend Technologies**

- Java 17 → Primary backend language.
- Spring Boot → Framework for creating the REST API.
- Spring Web → Handling of HTTP requests (GET, POST, etc.).
- Spring Data JPA → Database access and management.
- Hibernate → ORM used by JPA to map objects to tables.
- MySQL → Relational database where users, albums and collections are stored.
- Maven → Dependency management and build system.
- Discogs API → External service for obtaining real-time vinyl information.

- **Front-end Technologies**

- HTML5 → Interface structure.
- CSS3 → Styling and visual design.
- JavaScript → Client-side logic and communication with the back-end.
- Fetch API → Sending requests to the back-end (REST).
- LocalStorage → Managing user sessions in the browser.

- **Development tools**

- Visual Studio Code → The project's main editor.
- Live Server → A local server for running the frontend.
- PowerShell → A terminal used to run Maven and Spring Boot.
- MySQL Workbench → Visual database management.
- Postman → Testing REST API endpoints.

- GitHub → Version control and project storage.
 - **Key libraries and dependencies**
- spring-boot-starter-web → Controllers and REST endpoints.
- spring-boot-starter-data-jpa → Repositories and access to MySQL.
- mysql-connector-j → JDBC connector for MySQL.
- lombok → Code simplification (getters, setters, constructors).
- spring-boot-devtools → Automatic reloading during development.

System architecture

The VinylMarket system is designed using a layered architecture that clearly separates business logic, data access and the user interface. This structure facilitates maintenance, scalability and understanding of the project, allowing for organised work on both the backend and the frontend.

The architecture is divided into three main blocks: Frontend, Backend and Database, connected to one another via a REST API.

- **General system architecture**
- The system follows a Client–Server model, where:
 - The frontend acts as the client, running in the user's browser.
 - The backend acts as the server, processing requests and managing the logic.
 - The database stores the system's persistent data.
- The Discogs API functions as an external service to retrieve real-world vinyl data.
- The general flow is:
 - The user interacts with the frontend.
 - The frontend sends requests to the backend via the Fetch API.
 - The backend processes the request and accesses the database or Discogs.
 - The backend returns a response in JSON format.
 - The frontend updates the interface with the received data.
- **Backend Architecture (Spring Boot)**

The backend is organized following the **MVC (Model–View–Controller)** pattern adapted for REST APIs.

Model

Contains the entity classes representing the system's data:

- Disco
- Usuario
- Coleccion

These classes are mapped to MySQL tables using JPA/Hibernate.

Repository

Interfaces responsible for database access:

- ViniloRepository
- UsuarioRepository
- ColeccionRepository

They **provide CRUD operations without writing SQL manually.**

Service

Contains the business logic of the application:

- Importing vinyls from Discogs
- Validating users
- Managing collections
- Preventing duplicates

Main services:

- ViniloService
- UsuarioService
- ColeccionService
- DiscogsService

Controller

Exposes REST endpoints to the frontend:

- /api/vinilos
- /api/usuarios
- /api/coleccion

Controllers receive requests and return JSON responses.

- **Frontend Architecture**

The frontend consists of:

HTML

Defines the structure of the pages:

- index.html
- dashboard.html
- login.html
- register.html

CSS

Defines the visual design and layout.

JavaScript

Handles client-side logic:

- Fetch requests to the backend
- Dynamic rendering of vinyl cards
- Session management using localStorage
- Button events (search, import, add to collection)

Main script:

- app.js

- **Database Architecture (MySQL)**

The database contains three main tables:

- **usuarios** → stores user credentials and profile data
- **vinilos** → stores the general vinyl catalog
- **coleccion** → links users with the vinyls in their personal collection

Relationships:

One user can have many vinyls in their collection.

One vinyl can belong to multiple users.

- **Integration with the Discogs API**

The backend includes a dedicated service (DiscogsService) that:

- Sends HTTP requests to the Discogs API.
- Receives the response in JSON format.

- Converts the results into Vinilo objects.
- Returns them to the frontend or stores them in the database.

This allows the system to work with real vinyl data without storing everything locally.

Database Design

The **VinylMarket** database has been designed to efficiently store information related to users, vinyl records, and personal collections. The system uses **MySQL** as the database engine and follows a relational model to ensure data integrity, consistency, and scalability.

The design is based on three main tables: **users**, **vinilos**, and **coleccion**, which represent the core entities of the system.

- **Design Objectives**

The database design aims to achieve the following goals:

- Ensure **data persistence** for all system information.
- Prevent **duplicate records**, especially for vinyls imported from Discogs.
- Allow each user to maintain their **own personal collection**.
- Enable fast and efficient queries from the backend.
- Maintain a clear and scalable structure for future improvements.

- **Main Tables**

1. **Usuarios Table**

Stores the basic information of each registered user, along with additional profile data and collector preferences.

Main fields:

id (PK) — Unique identifier.

email — User email (unique).

password — Encrypted password.

nombre — User's first name.

username — Unique username for login and display.

rol — User role (USER, ADMIN). Default is "USUARIO".

apellido — User's last name (optional).

ciudad — City of residence (optional).

direccion — Street address (optional).

codigo_postal — Postal code (optional, 5 digits max).

telefono — Phone number (optional, 9 digits max).

prefijo — International prefix (optional, e.g., "+34", max 3 characters).

fecha_registro — Account creation timestamp (auto-generated).

tipo_cliente — User type: comprador, vendedor, or ambos (both). Default is "comprador".

valoracion_vendedor — Seller rating (0-5, only relevant for sellers).

tipo_coleccionista — Collector level: principiante (beginner), aficionado (enthusiast), coleccionista (collector), experto (expert). Default is "principiante".

Purpose:

Manage user authentication and access.

Store complete profile information.

Personalize the user experience (collector level, seller status).

Link to the user's personal vinyl collection.

2. Discos Table

Stores the vinyl records that belong to the general catalog.

Main fields:

- Id_vinilo (PK) — Unique identifier.
- Titulo — Vinyl title.
- Artista — Artist or band.
- Genero — Music genre.
- Anio — Release year.
- Imagen — Cover image URL.
- DiscogsId — Unique Discogs ID (prevents duplicates).

Purpose:

Represent the vinyl records available in the system.

3. coleccion Table

Links users with the vinyls they have added to their personal collection.

Main fields:

- Id (PK) — Unique identifier.
- Usuario_id (FK) — User who owns the record.

- Disco_id (FK) — Vinyl added to the collection.

Purpose:

Allow each user to maintain a personalized collection.

Prevent duplicates through a unique constraint (user + vinyl).

- **Table Relationships**

- One user can have many vinyls in their collection.
- One vinyl can belong to many users.
- The coleccion table acts as a linking table (simplified many-to-many relationship).

Relationship model:

- usuarios (1) — (N) coleccion (N) — (1) Vinilos

- **Normalization**

The database follows the **Third Normal Form (3NF)**:

- No duplicated data.
- Each field depends only on its primary key.
- Relationships are properly separated into independent tables.

This ensures a clean, efficient, and maintainable design.

- **Conceptual Diagram (explained)**

Although the visual diagram is not included here, the conceptual structure is:

User

- id
- email
- password

Vinilo

- id
- titulo
- artista
- genero
- anio
- imagen
- discogsId

Coleccion

- id
- usuario_id → User.id
- vinilo_id → Vinilo.id

Additional Tables Added to the Database

Two new tables were added to extend the functionality of the system:

Coleccion (user-owned vinyls) and **Lista_de_deseos** (wishlist). These tables expand the original design and allow users to manage both the vinyls they already own and those they want to acquire in the future.

Coleccion Table

The **Coleccion** table stores the vinyl records that a user has added to their personal collection. It represents a many-to-many relationship between *Usuarios* and *Vinilos*, resolved through this linking table.

Main fields:

- **id_coleccion** — Primary key
- **id_usuario** — User who owns the vinyl
- **id_vinilo** — Vinyl from the general catalogue
- **fecha_adquisicion** — Date the vinyl was added
- **estado** — Physical condition of the vinyl
- **calificacion** — User rating (1–5)

Purpose:

- Store vinyls owned by each user
- Prevent duplicates using a unique constraint
- Allow users to rate and track the condition of their vinyls

```
CREATE TABLE Coleccion (  
  id_coleccion INT AUTO_INCREMENT PRIMARY KEY,  
  id_cliente INT NOT NULL,  
  id_vinilo INT NOT NULL,  
  fecha_adquisicion DATE DEFAULT CURRENT_DATE,  
  estado ENUM('Nuevo', 'Como nuevo', 'Muy bueno', 'Bueno', 'Aceptable') DEFAULT  
  calificacion INT DEFAULT 5,  
  FOREIGN KEY (id_cliente) REFERENCES Clientes(id_cliente),  
  FOREIGN KEY (id_vinilo) REFERENCES Vinilos(id_vinilo),  
  UNIQUE (id_cliente, id_vinilo)  
);
```

Lista_de_deseos Table (Wishlist)

The **Lista_de_deseos** table stores vinyl records that a user wants to acquire in the future. It also represents a many-to-many relationship between *Usuarios* and *Vinilos*.

Main fields:

- **id_deseo** — Primary key
- **id_usuario** — User who added the vinyl to the wishlist
- **id_vinilo** — Vinyl from the catalogue
- **fecha_agregado** — Timestamp of when it was added
- **prioridad** — Priority level (High, Medium, Low)

Purpose:

- Allow users to maintain a wishlist
- Prevent duplicates
- Track interest level for each vinyl

```
CREATE TABLE Lista_de_deseos (  
  id_deseo INT AUTO_INCREMENT PRIMARY KEY,  
  id_cliente INT NOT NULL,  
  id_vinilo INT NOT NULL,  
  fecha_agregado DATETIME DEFAULT CURRENT_TIMESTAMP,  
  prioridad ENUM('Alta', 'Media', 'Baja') DEFAULT 'Media',  
  FOREIGN KEY (id_cliente) REFERENCES Clientes(id_cliente),  
  FOREIGN KEY (id_vinilo) REFERENCES Vinilos(id_vinilo),  
  UNIQUE (id_cliente, id_vinilo)  
);
```

Backend Modeling

The backend of **VinylMarket** is built using **Spring Boot** and follows the **MVC (Model–View–Controller)** architectural pattern adapted for REST APIs. This modeling defines how the code is structured, how classes interact, and how data flows from the database to the frontend.

- **Model Layer (Entities)**

The **Model** represents the main entities of the system. Each class corresponds to a database table and is managed through JPA/Hibernate.

Main entities:

- **Vinilo** Represents a vinyl record in the catalog. Fields: id, title, artist, genre, year, image, discogsId.

- **Usuario** Represents a registered user. Fields: id, email, password.
- **Coleccion** Represents the relationship between a user and a vinyl. Fields: id, usuario_id, id_vinilo.

Common annotations used:

- @Entity
- @Table
- @Id
- @GeneratedValue
- @ManyToOne
- @JoinColumn

- **Repository Layer**

Repositories are interfaces that provide access to the database without writing SQL manually.

Main repositories:

- **ViniloRepository** Extends JpaRepository<Vinilo, Long> Includes methods to search by discogsId.
- **UsuarioRepository** Extends JpaRepository<Usuario, Long> Includes methods to search by email.
- **ColeccionRepository** Extends JpaRepository<Coleccion, Long> Includes methods to retrieve a user's collection.

Repositories form the layer that communicates directly with MySQL.

- **Service Layer**

The **Service** layer contains the **business logic** of the application. This is where validations, rules, and operations between repositories are handled.

Main services:

ViniloService

- Import vinyls from Discogs
- Save vinyls to the database
- Prevent duplicates

- List catalog items

UsuarioService

- Register users
- Validate login credentials
- Check for duplicate emails

ColeccionService

- Add vinyls to a user's collection
- Prevent duplicates per user
- Retrieve personal collections

DiscogsService

- Connect to the Discogs API
- Process JSON responses
- Convert results into Vinilo objects

Common annotations:

- @Service
- @Autowired

- **Controller Layer**

Controllers expose the REST endpoints consumed by the frontend through Fetch API.

Main controllers:

ViniloController Endpoints:

- GET /api/vinilos
- GET /api/vinilos/buscar?q={query}
- POST /api/vinilos/import

UsuarioController Endpoints:

- POST /api/usuarios/register
- POST /api/usuarios/login

ColeccionController Endpoints:

- POST /api/coleccion/add
- GET /api/coleccion/{userId}

Common annotations:

- @RestController
- @RequestMapping
- @GetMapping
- @PostMapping

- **Advantages of This Modeling**

- Clean and maintainable code
- Clear separation of layers
- Easy to extend with new features
- Follows Spring Boot best practices
- Allows external API integration without affecting other layers

Backend Implementation

The backend of VinylMarket has been implemented using Spring Boot, following a layered architecture that separates entities, repositories, services, and controllers. This structure ensures clean code, maintainability, and efficient communication with the database and the Discogs API.

- **Project Configuration**

- Language: Java 17
- Framework: Spring Boot

- Build tool: Maven
- Main dependencies:
- spring-boot-starter-web → REST API creation
- spring-boot-starter-data-jpa → data access with JPA/Hibernate
- mysql-connector-j → MySQL JDBC driver
- lombok → reduces boilerplate code
- spring-boot-devtools → auto-reload during development

Database configuration is defined in application.properties, including URL, username, password, and Hibernate dialect.

- **Entity Implementation**

The entity classes are located in the model or entity package. Each class is mapped to a database table using JPA annotations.

Main entities:

- **Vinilo**
 - Annotations: @Entity, @Table(name = "viniloss")
 - Fields: id, title, artist, genre, year, image, discogsId
 - Primary key: @Id, @GeneratedValue
- **Usuario**
 - Annotations: @Entity, @Table(name = "usuarios")
 - Fields: id, email, password
- **Coleccion**
 - Annotations: @Entity, @Table(name = "coleccion")
 - Fields: id, usuario, vinilo
 - Relationships:
 - @ManyToOne with Usuario
 - @ManyToOne with Vinilo
 - @JoinColumn for foreign keys

Lombok annotations such as @Data, @NoArgsConstructor, and @AllArgsConstructor are used to simplify code.

- **Repository Implementation**

Repositories are interfaces located in the repository package. They extend JpaRepository to provide CRUD operations automatically.

Main repositories:

- **ViniloRepository**

- Extends JpaRepository<Vinilo, Long>
- Custom method:
 - Optional<Vinilo> findByDiscogsId(String discogsId)
- **UsuarioRepository**
 - Extends JpaRepository<Usuario, Long>
 - Custom method:
 - Optional<Usuario> findByEmail(String email)
- **ColeccionRepository**
 - Extends JpaRepository<Coleccion, Long>
 - Custom methods:
 - List<Coleccion> findByUsuariold(Long usuariold)

Repositories handle all communication with MySQL.

- **Service Implementation**

The **service layer** contains the business logic and coordinates operations between repositories and external APIs.

Main services:

- **ViniloService**
 - Checks for duplicates using discogsId
 - Saves vinyls to the database
 - Lists catalog items
 - Works with DiscogsService to import vinyls
- **UsuarioService**
 - Registers new users
 - Validates login credentials
 - Checks for duplicate emails
- **ColeccionService**
 - Adds vinyls to a user's collection
 - Prevents duplicates per user
 - Retrieves a user's collection
- **DiscogsService**
 - Sends HTTP requests to the Discogs API
 - Parses JSON responses
 - Converts results into Vinilo objects

Services use @Service and dependency injection via constructors or @Autowired.

- **Controller Implementation**

Controllers expose REST endpoints that the frontend consumes using Fetch API.

Main controllers:

- **ViniloController** Endpoints:

- GET /api/vinilos — returns the catalog
- GET /api/vinilos/search — searches Discogs
- POST /api/vinilos/import — imports a vinyl
- **UsuarioController** Endpoints:
 - POST /api/usuarios/register — user registration
 - POST /api/usuarios/login — login validation
- **ColeccionController** Endpoints:
 - POST /api/coleccion/add — adds a vinyl to a user's collection
 - GET /api/coleccion/{userId} — retrieves a user's collection

Controllers use:

- @RestController
- @RequestMapping
- @GetMapping
- @PostMapping
- @RequestBody
- @PathVariable

- **Testing and Execution**

- The backend is executed using:
 - mvn spring-boot:run
 - or the generated .jar file
- Endpoints were tested with:
 - **Postman**
 - **Frontend Fetch API**
- Verified functionalities:
 - User registration and login
 - Discogs search
 - Vinyl import
 - Catalog listing
 - Personal collection management

Frontend Modeling

The frontend of **VinylMarket** is designed as a lightweight web application built with **HTML, CSS, and JavaScript**, consuming the REST API provided by the backend. Its main goal is to offer a clean, intuitive interface where users can register, log in, search vinyls, import them from Discogs, and manage their personal collection.

The frontend is structured into multiple HTML pages, a main JavaScript file, and supporting CSS stylesheets.

- **General Structure of the Frontend**

The frontend consists of three main components:

- **HTML pages** — define the structure of each view.
- **CSS stylesheets** (**styles.css**, **base.css**, **navbar.css**, **sidebar.css**, **auth.css**, **dashboard.css**, **perfil.css**, **components.css**, **responsive.css**)— define the visual appearance.
- **Various JavaScript files** (**config.js**, **auth.js**, **dashboard.js**, **index.js**, **perfil.js**, **notifications.js**, **utils.js**) — handles logic, API communication, and dynamic rendering.

This separation ensures clarity, maintainability, and scalability.

- **Main Pages**

index.html

- Entry point of the application.
- Can display introductory information or redirect to login/registration.

login.html

- Contains the **login form**.
- Fields: email and password.
- Sends a request to the backend to validate credentials.
- Redirects to dashboard.html upon successful login.

register.html

- Contains the **user registration form**.
- Fields: email and password.
- Sends data to the backend to create a new user.
- Displays error messages if the email already exists.

dashboard.html

- Main page after logging in.
- Displays:
 - Discogs search results
 - The general vinyl catalog
 - The user's personal collection
- Includes buttons for:
 - Searching vinyls
 - Importing vinyls
 - Adding vinyls to the collection
 - Logging out
- **Frontend Logic (JavaScript)**

All core logic is implemented in:

- app.js

Main responsibilities:

- **Session Management**
 - Store logged-in user in localStorage
 - Check session on page load
 - Redirect to login if no session exists
 - Clear session on logout
- **Backend Communication**
 - Uses **Fetch API** to call:
 - /api/usuarios/login
 - /api/usuarios/register
 - /api/vinilos
 - /api/vinilos/search
 - /api/vinilos/import
 - /api/coleccion/add
 - /api/coleccion/{userId}
 - Sends and receives data in **JSON** format.
- **Dynamic Rendering**
 - Creates vinyl cards (title, artist, image, buttons).
 - Displays:
 - Discogs search results
 - The general catalog
 - The user's personal collection
- **Event Handling**
 - Click events for:
 - Search
 - Import
 - Add to collection
 - Logout
- **Styles and Design (CSS)**

CSS files define:

- Colors, fonts, spacing
- Layout of vinyl cards
- Form styling (login, registration)
- Dashboard layout (search, catalog, collection sections)

The goal is to provide a clean, modern, and user-friendly interface.

- **Frontend–Backend Interaction**

The frontend acts purely as a **client**, meaning:

- It sends requests to the backend
- Receives JSON responses
- Updates the UI accordingly

This ensures:

- The backend handles business logic and data management
- The frontend focuses on user experience and presentation

Frontend Implementation

The frontend of **VinylMarket** is implemented using **HTML, CSS, and JavaScript**, structured to interact with the backend through REST API calls. Its main responsibilities include handling user actions, managing sessions, sending requests, and dynamically rendering data.

Structure

The frontend is organized into:

- **HTML pages** (index.html, login.html, register.html, dashboard.html)
- **CSS styles** for layout and design
- app.js for logic, API communication, and UI updates

HTML Pages

Each page serves a specific purpose:

- **index.html** — Entry point
- **login.html** — Login form, sends credentials to backend
- **register.html** — Registration form, handles duplicate email errors
- **dashboard.html** — Main interface showing:
 - Discogs search results
 - General catalog
 - User collection
 - Buttons for search, import, add to collection, logout

JavaScript Logic (app.js)

Main features:

- **Session management** using localStorage
- **Fetch API** calls to backend endpoints
- **Dynamic rendering** of vinyl cards, search results, catalog, and collection
- **Event handling** for search, import, add, and logout

CSS Styling

CSS defines:

- Layout of vinyl cards

- Dashboard grid
- Form styling
- Colors, spacing, and typography

Error Handling

Includes:

- Alerts for invalid login
- Messages for duplicate registration
- Warnings for empty Discogs results

System Testing

System testing was performed to verify that the entire VinylMarket application works correctly as a unified system. The goal was to ensure that all components—frontend, backend, database, and the Discogs API—interact properly and meet the project requirements.

Testing Objectives

The main objectives of system testing were:

- Validate that all functional requirements are correctly implemented.
- Ensure proper integration between frontend and backend.
- Confirm that communication with the Discogs API works reliably.
- Detect and correct errors before deployment.
- Verify that the system behaves correctly under normal usage conditions.

Test Environment

Testing was conducted in the following environment:

- Backend running locally with Spring Boot.
- MySQL database.
- Frontend executed in a web browser.
- Tools used:
 - Postman for API testing.
 - Browser Developer Tools for debugging.
 - MySQL Workbench for database inspection.

Functional Tests

The following functionalities were tested:

User Management

- User registration with valid and duplicate emails.
- User login with correct and incorrect credentials.
- Session persistence using localStorage.

Vinyl Catalog

- Retrieval of all stored vinyls.
- Correct JSON formatting.

Discogs Search

- Searches with valid terms.
- Handling of empty results.
- Proper processing of API responses.

Vinyl Import

- Successful import of vinyls.
- Duplicate prevention using discogsId.

User Collection

- Adding vinyls to a user's collection.
- Preventing duplicate entries.
- Retrieving the user's complete collection.

Integration Tests

Integration tests validated communication between:

- Frontend and backend (Fetch API responses).
- Backend and database (correct saving and querying).
- Backend and Discogs API (JSON parsing and error handling).

User Interface Tests

The UI was tested to ensure:

- Buttons trigger the correct actions.
- Vinyl cards render properly.
- Forms validate user input.
- Dashboard updates dynamically.
- Layout remains consistent.

Error Handling Tests

The system was tested against:

- Invalid login attempts.
- Duplicate user registration.
- Discogs API returning no results.
- Backend or database connection failures.

The frontend displays clear error messages and prevents crashes.

Test Results

All critical functionalities worked correctly:

- User registration and login: OK
- Discogs search and import: OK
- Catalog and collection management: OK
- Frontend-backend integration: OK
- Error handling: OK

Communication Between Frontend and Backend

The communication between the frontend and backend in **VinylMarket** is based on a simple REST architecture. The frontend uses **JavaScript (Fetch API)** to send HTTP requests to the backend, and all data is exchanged in **JSON format**.

1. How Communication Works

1. The user performs an action in the frontend.
2. JavaScript sends a **GET** or **POST** request to the backend.
3. The backend processes the request using controllers and services.
4. The backend interacts with the database or the Discogs API if needed.
5. A JSON response is returned to the frontend.
6. The frontend updates the interface dynamically.

2. HTTP Methods

- **GET** → Retrieve data (catalog, collection, Discogs search).
- **POST** → Send data (login, register, import vinyls, add to collection).

3. JSON Data Exchange

Both sides use JSON for requests and responses, making parsing and rendering simple.

4. Frontend Requests

The frontend uses the **Fetch API** to call backend endpoints such as:

- `/api/usuarios/login`
- `/api/vinilos/search`
- `/api/coleccion/add`

Example:

```
javascript
fetch("/api/usuarios/login", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ email, password })
});
```

5. Backend Controllers

The backend exposes REST endpoints using:

- @RestController
- @GetMapping
- @PostMapping

Each controller handles a specific part of the system (users, vinyls, collections).

6. Session Handling

The backend is stateless. The frontend stores the logged-in user in **localStorage** to maintain the session.

7. Error Handling

- The frontend shows alerts for invalid login, duplicate registration, or empty search results.
- The backend returns HTTP status codes and JSON error messages.

User Manual

This user manual explains how to use the **VinylMarket** application, including how to register, log in, search for vinyls, import them from Discogs, and manage the personal collection.

Accessing the Application

To access the application, the user must open index.html in a web browser. From this page, the user can navigate to:

- **Login page**
- **Registration page**

User Registration

If the user does not have an account, they must register.

Steps:

1. Open **register.html**
2. Enter:
 - Email
 - Password
3. Click “**Register**”
4. If the email already exists, an error message will appear

After registration, the user can log in.

Login

To access the system:

1. Open **login.html**
2. Enter email and password
3. Click “**Login**”

If the credentials are correct, the user is redirected to the **dashboard**. If not, an error message is displayed.

Main Dashboard

The **dashboard** is the main page after logging in. It contains three sections:

- **Discogs Search**
- **General Vinyl Catalog**
- **User Collection**

It also includes a **Logout** button.

Searching Vinyls on Discogs

The user can search for vinyls using the Discogs API.

Steps:

1. Enter an artist or album name in the search bar
2. Click “**Search**”
3. Results appear as cards showing:
 - Title
 - Artist
 - Cover image
 - “**Import**” button

If no results are found, a warning is displayed.

Importing Vinyls from Discogs

To add a vinyl to the internal catalog:

1. Perform a search
2. Click “**Import**” on the desired vinyl
3. The vinyl is saved in the database
4. It automatically appears in the **general catalog**

Duplicate imports are prevented.

Adding Vinyls to the User Collection

The user can add any vinyl from the catalog to their personal collection.

Steps:

1. Go to the **general catalog**
2. Click “**Add to collection**”
3. The vinyl appears in **My Collection**

Duplicates are not allowed.

Viewing the User Collection

In **My Collection**, the user can see all vinyls they have added.

Each card displays:

- Cover image
- Title
- Artist

The section updates automatically.

Logging Out

To log out:

1. Click the “**Logout**” button
2. The session stored in the browser is removed
3. The user is redirected to the login page

Common Issues and Solutions

- **Cannot log in**
 - Check email and password
 - Ensure the user is registered
- **No Discogs results**
 - Check internet connection
 - Try a different search term
- **Vinyl does not import**
 - It may already exist in the catalog

Project Conclusions

The development of **VinylMarket** has made it possible to create a complete and functional system for managing vinyl records through a combination of frontend, backend, database, and external API integration. The project successfully meets the objectives defined at the beginning and demonstrates a solid understanding of full-stack application development.

The backend, built with Spring Boot, provides a stable and well-structured REST API that handles user management, catalog operations, and Discogs integration. The database design ensures consistent storage of vinyls and user collections, avoiding duplicates and maintaining data integrity.

The frontend, implemented with HTML, CSS, and JavaScript, offers a simple and intuitive interface that allows users to register, log in, search for vinyls, import them, and manage their personal collection. Communication between frontend and backend is efficient thanks to JSON-based requests using the Fetch API.

Throughout the project, different types of testing confirmed that the system works correctly as a whole. Functional tests, integration tests, and user interface tests validated the correct behavior of each component and the interaction between them.

In conclusion, **VinylMarket** achieves its goal of providing a practical and user-friendly platform for vinyl management. The project demonstrates the student's ability to design, implement, and test a complete full-stack application, integrating external services and applying good development practices.

Bibliography / Sources

This section lists all the sources, tools, and references used during the development of the VinylMarket project. They include official documentation, APIs, frameworks, and learning resources consulted throughout the implementation process.

Official Documentation

- **Spring Boot Documentation** – Reference for building REST APIs, controllers, services, and project configuration.
- **Java Documentation** – Used for understanding language features and standard libraries.
- **MySQL Documentation** – Reference for SQL queries, database configuration, and table design.
- **Discogs API Documentation** – Used to integrate external vinyl search and import functionality.
- **HTML/CSS MDN Web Docs** – Reference for frontend structure and styling.

- **JavaScript MDN Web Docs** – Used for Fetch API, DOM manipulation, and event handling.

Tools and Software

- **Postman** – Used for testing backend endpoints.
- **MySQL Workbench** – Used for database visualization and query testing.
- **Visual Studio Code** – Main editor for frontend development.
- **IntelliJ IDEA** – Main IDE for backend development.

Learning Resources

- **Stack Overflow** – Used to resolve specific implementation doubts.
- **W3Schools** – Reference for basic HTML, CSS, and JavaScript examples.
- **Baeldung** – Tutorials for Spring Boot and REST API development.

Additional References

- **GitHub** – Used for version control and repository management.
- **JSON.org**